# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE CODE: CSA51**
**COURSE NAME: Cryptography and Network Security**

---

**COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

**Analytical Problem Solving: 40%**

**QUESTIONS:** **Total Marks: 50**

**Annexure A**

---

***Code of Conduct:***

*I K . B h a v a n a ( 1 9 2 5 2 4 1 7 8 ) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***
***K.Bhavana***

# Question 1: RSA Key Generation and Security Analysis

## Given: p = 17, q = 19, e = 5, Message M = 7

### 1.1 Introduction to RSA and Public Key Cryptography

RSA (Rivest–Shamir–Adleman) is the most widely used asymmetric (public-key) cryptographic algorithm. Unlike symmetric algorithms such as AES, where the same secret key is used for both encryption and decryption, RSA uses a mathematically related pair of keys: a public key that anyone can use to encrypt a message or verify a signature, and a private key, known only to the owner, used to decrypt messages or generate signatures.

The security of RSA rests on a one-way trapdoor function: it is computationally easy to multiply two large prime numbers together to obtain their product n, but it is computationally infeasible (with classical computers, given large enough primes) to reverse the process — that is, to factor n back into its two prime components. This asymmetry between the ease of multiplication and the difficulty of factorization is what allows the public key to be shared openly while the private key remains secure.

RSA relies on several results from number theory, most importantly Euler's theorem and the concept of modular multiplicative inverses. The algorithm consists of three main phases: key generation, encryption, and decryption. Each of these phases is demonstrated numerically below using the given values of p and q.

### 1.2 Given Parameters

| Parameter | Value | Description |
|---|---|---|
| p | 17 | First prime number |
| q | 19 | Second prime number |
| e | 5 | Chosen public exponent |
| M | 7 | Plaintext message to encrypt |

### 1.3 Mathematical Foundations Used in This Solution

Before working through the numeric steps, it is worth reviewing the core number-theory results that make RSA possible, since every step below relies on one of these ideas.

#### 1.3.1 Modular Arithmetic

All RSA operations take place in modular arithmetic, where numbers 'wrap around' after reaching a fixed modulus n. Formally, a ≡ b (mod n) means that a and b leave the same remainder when divided by n. Modular arithmetic preserves addition, subtraction, and multiplication, which is what allows large exponentiations like M^e to be computed step-by-step while keeping every intermediate value bounded below n, rather than computing an astronomically large number first and reducing it at the end.

### 1.3.2 Euler's Totient Function

For any positive integer n, φ(n) is defined as the count of integers in the range 1 to n that are coprime to n. When n is the product of two distinct primes p and q, this count simplifies neatly to φ(n) = (p−1)(q−1), because the only integers in [1, n] that are NOT coprime to n are the multiples of p (there are q−1 of them, excluding n itself) and the multiples of q (p−1 of them), with no overlap since p and q are distinct primes. This is why RSA specifically requires n to be a product of exactly two distinct primes — it is what makes φ(n) easy to compute for the key generator (who knows p and q) yet hard to compute for an attacker (who only sees n).

### 1.3.3 Euler's Theorem and Why Decryption Works

Euler's theorem states that for any integer a coprime to n, a^φ(n) ≡ 1 (mod n). Because d is chosen so that e·d ≡ 1 (mod φ(n)), we can write e·d = 1 + k·φ(n) for some integer k. Then for any message M coprime to n:

```
(M^e)^d = M^(e•d) = M^(1 + k•φ(n)) = M • (M^φ(n))^k ≡ M • 1^k ≡ M (mod n)
```

This algebraic identity is the entire reason RSA decryption recovers the original plaintext exactly — it is not a coincidence or an approximation, but a guaranteed consequence of how e and d were constructed relative to φ(n). This is what Step 4 verifies numerically.

### 1.3.4 The Extended Euclidean Algorithm

Because d must satisfy e·d ≡ 1 (mod φ(n)), finding d is equivalent to finding the modular multiplicative inverse of e with respect to φ(n). The Extended Euclidean Algorithm computes this efficiently by tracking, at each step of the ordinary Euclidean (GCD) algorithm, how the remainder can be expressed as a linear combination of the two original numbers. This is applied concretely in Step 2 below.

## 1.4 Step 1: Computing the Modulus n and Euler's Totient φ(n)

The modulus n forms part of both the public and private keys and defines the size of the field within which all RSA arithmetic is performed. It is computed simply as the product of the two chosen primes:

**n = p × q** = 17 × 19 = 323

Euler's totient function φ(n) counts the number of positive integers less than n that are coprime to n (i.e., share no common factor with n other than 1). For a modulus formed from two distinct primes, φ(n) has a convenient closed form:

**φ(n) = (p − 1)(q − 1) =** 16 × 18 = 288

This value of φ(n) = 288 is critical because, by Euler's theorem, for any integer a coprime to n, a^φ(n) ≡ 1 (mod n). This identity is what makes RSA encryption and decryption mathematically invertible, and φ(n) is used directly in the next step to compute the private exponent d. Note that φ(n) must be kept secret (or is discarded after key generation), since anyone who knows φ(n) together with n can derive the private key exactly as we do below.

### 1.5 Step 2: Selecting e and Computing the Private Key d

The public exponent e must satisfy two conditions: 1 < e < φ(n), and gcd(e, φ(n)) = 1 (e must be coprime to φ(n), so that a modular inverse exists). Here e = 5 is given. We verify gcd(5, 288) = 1 using the Euclidean algorithm: 288 = 57×5 + 3; 5 = 1×3 + 2; 3 = 1×2 + 1; 2 = 2×1 + 0. The last non-zero remainder is 1, confirming e and φ(n) are coprime, so a valid d exists.

The private key d is the modular multiplicative inverse of e modulo φ(n), i.e. the unique integer satisfying:

**(e × d) mod φ(n) = 1** ⇒ 5d ≡ 1 (mod 288)

We solve this using the Extended Euclidean Algorithm, which works backward through the Euclidean division steps to express 1 as a linear combination of 288 and 5:

| Step | Equation | Remainder |
|---|---|---|
| 1 | 288 = 57 × 5 + 3 | 3 |
| 2 | 5 = 1 × 3 + 2 | 2 |
| 3 | 3 = 1 × 2 + 1 | 1 |
| 4 | 2 = 2 × 1 + 0 | 0 (stop) |

Back-substituting from the second-to-last line upward:

```
1 = 3 − 1(2)
```

$= 3 - 1(5 - 1\times3) = 2(3) - 1(5)$

$= 2(288 - 57\times5) - 1(5) = 2(288) - 115(5)$

This gives $-115(5) \equiv 1 \pmod{288}$, so $d \equiv -115 \pmod{288} \equiv 288 - 115 = 173$.

**Private key d =** 173

**Verification:** $5 \times 173 = 865 = 3 \times 288 + 1$ ✓ (remainder is exactly 1, confirming d is correct)

| Key | Form | Value |
|---|---|---|
| Public Key | (e, n) | (5, 323) |
| Private Key | (d, n) | (173, 323) |

## 1.6 Step 3: Encrypting the Message M = 7

Encryption in RSA is performed by raising the plaintext message to the power of the public exponent e, modulo n:

**C = M^e mod n =** 7^5 mod 323

Direct computation of 7^5 = 16807 is possible for small numbers, but RSA in practice uses the 'square-and-multiply' method to keep intermediate values small, which is demonstrated below:

| Computation | Result | Reduced mod 323 |
|---|---|---|
| $7^1$ | 7 | 7 |
| $7^2$ | 49 | 49 |
| $7^3 = 7^2 \times 7$ | 343 | $343 - 323 = 20$ |
| $7^4 = 7^3 \times 7$ | $20 \times 7 = 140$ | 140 |
| $7^5 = 7^4 \times 7$ | $140 \times 7 = 980$ | $980 - (2\times323) = 334 \rightarrow 334-323=11$ |

**Ciphertext C =** 11

This value, C = 11, is the encrypted form of the message and is what would be

transmitted over an insecure channel. Without knowledge of the private key d, an eavesdropper who intercepts C = 11 cannot feasibly recover M = 7, other than by brute-force search over the small message space used in this illustrative example (which is only possible because n = 323 is deliberately tiny for teaching purposes).

## 1.7 Step 4: Decrypting the Ciphertext and Verifying Correctness

Decryption reverses encryption using the private exponent d: M = C^d mod n = 11^173 mod 323. Computing 11^173 directly is impractical by hand, so we exploit the fact that n = p × q and apply the Chinese Remainder Theorem (CRT), which is also how real RSA implementations accelerate decryption (roughly a 4× speed-up in practice).

### CRT Method

- Reduce the exponent modulo (p − 1) and (q − 1):

**d mod (p − 1) =** 173 mod 16 = 13

**d mod (q − 1) =** 173 mod 18 = 11

- Compute the partial result modulo each prime separately:

Mp = C^13 mod 17: $11^2 = 121 \bmod 17 = 2$; $11^4 = 2^2 = 4$; $11^8 = 4^2 = 16$; $11^{13} = 11^8 \times 11^4 \times 11^1 = 16 \times 4 \times 11 \bmod 17 = 704 \bmod 17 = 7$ → Mp = 7

Mq = C^11 mod 19: $11^2 = 121 \bmod 19 = 7$; $11^4 = 7^2 = 49 \bmod 19 = 11$; $11^8 = 11^2 \bmod 19 = 7$; $11^{11} = 11^8 \times 11^2 \times 11^1 = 7 \times 7 \times 11 \bmod 19 = 539 \bmod 19 = 7$ → Mq = 7

- Recombine using CRT: since both Mp = 7 and Mq = 7, and 7 is already less than both primes, the unique solution modulo n = 323 is simply:

**Recovered message M =** 7

### Verification

The decrypted value M = 7 exactly matches the original plaintext message that was encrypted in Step 3. This confirms that the key pair (e, d) generated in Step 2 is mathematically consistent, and that the encryption/decryption round-trip (M → C → M) is lossless, which is the defining correctness property required of any valid RSA key pair.

## 1.8 Step 5: Impact of Choosing Small Prime Numbers on RSA Security

Although the numbers used above (p = 17, q = 19) are excellent for illustrating the mechanics of RSA by hand, they would be catastrophically insecure in any real deployment. This section analyses why.

### 1.7.1 The Factorization Attack

The entire security of RSA is founded on the assumption that factoring n = p × q is computationally infeasible when p and q are large. With n = 323, however, an attacker needs only test odd divisors up to $\sqrt{323} \approx 18$ to find p = 17 by trial division — a task a calculator (or even mental arithmetic) can complete in seconds. Once n is factored, the attacker immediately recovers φ(n) = (p−1)(q−1) and, using the same Extended Euclidean Algorithm shown in Step 2,

derives the private key d from the public key e. The entire cryptosystem is broken with no advanced tools required.

### 1.7.2 Why Key Size Matters

Modern factorization algorithms such as the General Number Field Sieve (GNFS) scale sub-exponentially with the size of n, meaning that as the bit-length of n grows, factoring becomes exponentially harder relative to the effort of key generation and encryption. This is why standards bodies mandate minimum key sizes:

| RSA Key Size (bits) | Approximate Security Level | Current Status |
|---|---|---|
| 512 | ~ trivial (broken) | Factored in hours on cloud hardware; unsafe |
| 1024 | ~ 80-bit security | Considered weak / deprecated |
| 2048 | ~ 112-bit security | Current minimum industry standard |
| 3072 | ~ 128-bit security | Recommended for long-term security |
| 4096 | ~ 150-bit security | Used for high-security / long-lived keys |

### 1.7.3 Additional Risks from Small or Poorly Chosen Primes

- Close primes: if p and q are close in value, Fermat's factorization method can recover them almost instantly, regardless of n's bit length.
- Shared prime reuse: if the same prime is accidentally reused across two different moduli (a real-world flaw found in some embedded devices), computing gcd(n1, n2) instantly reveals the shared factor.
- Small message space with small n: as seen in this example, a small n also limits the possible plaintext values, making brute-force or lookup-table attacks feasible even without factoring n directly.

### 1.7.4 Practical Recommendation

For genuine security, p and q must each be randomly generated primes of at least 1024 bits (giving n of 2048 bits or more), chosen independently, tested for primality using probabilistic tests such as Miller–Rabin, and kept sufficiently far apart in magnitude to resist Fermat-style attacks. The p = 17, q = 19 example used here is pedagogically valuable precisely because its small size lets every step be verified by hand — but it also visibly demonstrates, through the ease of the attack described above, exactly why real systems cannot use small primes.

### 1.8.5 Historical Record: Real RSA Factorization Challenges

The RSA Factoring Challenge, run between 1991 and 2007, published a series of RSA moduli of increasing size and offered cash prizes for factoring them, giving the cryptographic community concrete, dated evidence of how key size relates to real-world security.

| Challenge | Key Size (bits) | Factored In |
|---|---|---|

| | | |
|---|---|---|
| RSA-100 | 330 | 1991 (days) |
| RSA-512 | 512 | 1999 (~7 months, later minutes with modern hardware) |
| RSA-768 | 768 | 2009 (~2 years of distributed computation) |
| RSA-2048 | 2048 | Not factored to date — current recommended minimum |

This progression shows that key sizes considered secure in one decade (512-bit, 768-bit) were later factored as computing power grew, which is exactly why standards bodies such as NIST periodically revise minimum recommended key lengths upward, and why the p = 17, q = 19 example in this question — equivalent to roughly a 9-bit modulus — would not even register as a meaningful challenge; it can be factored by inspection.

### 1.9 Additional Cross-Check: Direct Binary (Square-and-Multiply) Verification

As a further check, Step 4 can also be verified directly — without using the CRT shortcut — using the standard square-and-multiply algorithm that real RSA implementations use to compute modular exponentiation efficiently. The exponent d = 173 is first converted to binary:

**173 in binary** = 1 0 1 0 1 1 0 1  (128 + 32 + 8 + 4 + 1 = 173)

Processing the bits from most significant to least significant, the running result is squared at every bit, and additionally multiplied by the base (11) whenever the current bit is 1:

| Bit # | Bit value | Running value after squaring | Running value after multiply (if bit=1) |
|---|---|---|---|
| 1 (MSB) | 1 | $1^2 \bmod 323 = 1$ | $1 \times 11 \bmod 323 = 11$ |
| 2 | 0 | $11^2 \bmod 323 = 121$ | — (no multiply) |
| 3 | 1 | $121^2 \bmod 323 = 106$ | $106 \times 11 \bmod 323 = 197$ |
| 4 | 0 | $197^2 \bmod 323 = 49$ | — (no multiply) |
| 5 | 1 | $49^2 \bmod 323 = 140$ | $140 \times 11 \bmod 323 = 248$ |
| 6 | 1 | $248^2 \bmod 323 = 134$ | $134 \times 11 \bmod 323 = 182$ |
| 7 | 0 | $182^2 \bmod 323 = 178$ | — (no multiply) |
| 8 (LSB) | 1 | $178^2 \bmod 323 = 30$ | $30 \times 11 \bmod 323 = 7$ |

Each row's 'running value' becomes the input to the next row's squaring step. After processing all 8 bits of the exponent, the algorithm arrives at:

**Final result:** 11^173 mod 323 = 7

This matches the CRT-based result from Step 4 exactly, giving an independent confirmation that the recovered plaintext M = 7 is correct through two different computational methods.

## 1.10 Summary of Results — Question 1

| Quantity | Value |
|---|---|
| n | 323 |
| φ(n) | 288 |
| Public key (e, n) | (5, 323) |
| Private key (d, n) | (173, 323) |
| Plaintext M | 7 |
| Ciphertext C | 11 |
| Decrypted message | 7 (matches original ✓) |

## 1.11 Conclusion

This exercise walked through the complete RSA lifecycle — key generation, encryption, and decryption — using small, hand-verifiable primes, and confirmed that the algorithm correctly recovers the original message M = 7 through the ciphertext C = 11. At the same time, the security analysis demonstrates that the same small size which makes the arithmetic tractable by hand also makes the scheme trivially breakable, underscoring that RSA's security is a direct function of key size and the computational hardness of integer factorization, not of the algorithm's structure alone.

# Question 3: RSA Digital Signature Verification

## Given: p = 11, q = 17, e = 7, Message M = 8

### 3.1 Introduction to Digital Signatures

A digital signature is the cryptographic equivalent of a handwritten signature or a stamped seal, but it offers far stronger guarantees. Where RSA encryption uses the recipient's public key to protect confidentiality, RSA digital signatures work in the opposite direction: the sender uses their own private key to 'sign' a message, and anyone can verify that signature using the sender's public key.

Because only the holder of the private key could have produced a valid signature, and because the signature is mathematically tied to the exact content of the message, digital signatures provide three essential security properties simultaneously: authentication (proof of who sent the message), integrity (proof the message was not altered), and non-repudiation (the sender cannot later deny having signed it). These properties make digital signatures foundational to technologies such as TLS/SSL certificates, code signing, e-government documents, and blockchain transactions.

The RSA signature scheme illustrated here is the 'textbook' version, where signing is modelled as decryption (using the private key) and verification as encryption (using the public key) — the reverse roles compared to standard RSA encryption/decryption of confidential data.

### 3.2 Given Parameters

| Parameter | Value | Description |
|---|---|---|
| p | 11 | First prime number |
| q | 17 | Second prime number |
| e | 7 | Public verification exponent |
| M | 8 | Message to be signed |

### 3.3 Mathematical Foundations of RSA Signatures

#### 3.3.1 Signing as Inverse Encryption

Textbook RSA signing deliberately swaps the roles of the two keys compared with encryption. In encryption, the sender uses the recipient's public key so that only the recipient's private key can

reverse it. In signing, the sender uses their own private key, so that the operation could only have been performed by someone holding that key; verification then uses the corresponding public key, which by definition anyone may access. This role-reversal is what turns the same trapdoor mathematics used for confidentiality into a mechanism for authenticity.

### 3.3.2 Why Real Systems Sign a Hash, Not the Message Itself

In this exercise, for simplicity and to match the question as given, the small message M = 8 is signed directly. In real-world RSA signature schemes (e.g., PKCS#1 v1.5 or PSS), however, the message is first passed through a cryptographic hash function such as SHA-256 to produce a fixed-length digest, and it is this digest — not the raw message — that is signed. This matters for two reasons: first, RSA signing is computationally expensive for large messages, so hashing first is far more efficient; second, signing the raw message directly (as in the simplified 'textbook RSA' shown here) is vulnerable to existential forgery attacks, where an attacker can construct valid (message, signature) pairs without ever knowing the private key, by exploiting the multiplicative structure of RSA (S1 × S2 corresponds to a signature on M1 × M2 mod n). Hashing breaks this multiplicative relationship and closes the attack.

### 3.3.3 Why the Same Key Pair Should Not Be Reused for Both Signing and Encryption

A related best practice is that a single RSA key pair should not be used for both encrypting messages and signing them, since a party willing to decrypt arbitrary ciphertexts sent to them could sometimes be tricked into producing a signature on data chosen by an attacker, and vice versa. In production systems, separate key pairs are generated for each purpose.

### 3.3.4 Common Hash Functions Paired with RSA Signatures

In production hash-then-sign schemes, the choice of hash function matters as much as the choice of key size, since the overall security of the signature can never exceed the collision resistance of the underlying hash. The table below summarises hash functions commonly paired with RSA in real systems:

| Hash Function | Digest Size | Security Status |
|---|---|---|
| MD5 | 128 bits | Broken — collisions found; must not be used |
| SHA-1 | 160 bits | Deprecated — practical collision attacks demonstrated |
| SHA-256 | 256 bits | Current widely-used standard (SHA-2 family) |
| SHA-3-256 | 256 bits | Modern alternative with a different internal design |

Because a hash collision (two different messages producing the same digest) would let an attacker substitute one signed message for another without detection, RSA signature schemes are only as trustworthy as the hash function feeding into them — which is why cryptographic guidance updates the recommended hash function over time, independently of RSA key size

recommendations.

### 3.4 Step 1: Computing the Modulus n and φ(n)

**n = p × q** = 11 × 17 = 187

**φ(n) = (p − 1)(q − 1)** = 10 × 16 = 160

As in RSA encryption, n = 187 defines the working modulus shared publicly, while φ(n) = 160 is used only internally during key generation to compute the private signing exponent d.

### 3.5 Step 2: Finding the Private Key d

The signer's private key d must satisfy 7d ≡ 1 (mod 160). We solve this with the Extended Euclidean Algorithm:

| Step | Equation | Remainder |
|---|---|---|
| 1 | 160 = 22 × 7 + 6 | 6 |
| 2 | 7 = 1 × 6 + 1 | 1 |
| 3 | 6 = 6 × 1 + 0 | 0 (stop) |

Back-substituting:

```
1 = 7 − 1(6)

  = 7 − 1(160 − 22×7) = 23(7) − 1(160)
```

So 23(7) ≡ 1 (mod 160), giving:

**Private key d** = 23

**Verification:** 7 × 23 = 161 = 1 × 160 + 1 ✓

| Key | Form | Value |
|---|---|---|
| Public Key (for verification) | (e, n) | (7, 187) |
| Private Key (for signing) | (d, n) | (23, 187) |

### 3.6 Step 3: Generating the Digital Signature

To sign the message, the sender computes S = M^d mod n using their private key — this is the reverse operation of standard RSA encryption, applied to the message itself rather than to a separately encrypted ciphertext:

**S = M^d mod n** = 8^23 mod 187

As before, we accelerate this using CRT with p = 11 and q = 17:

**d mod (p − 1)** = 23 mod 10 = 3

**d mod (q − 1)** = 23 mod 16 = 7

Sp = $8^3$ mod 11: $8^2 = 64 \bmod 11 = 9$; $8^3 = 9 \times 8 = 72 \bmod 11 = 6$ → Sp = 6

Sq = $8^7$ mod 17: $8^2 = 64 \bmod 17 = 13$; $8^4 = 13^2 = 169 \bmod 17 = 16$; $8^7 = 8^4 \times 8^2 \times 8^1 = 16 \times 13 \times 8 \bmod 17 = 1664 \bmod 17 = 15$ → Sq = 15

Recombining Sp = 6 (mod 11) and Sq = 15 (mod 17) via CRT: we seek S = 17k + 15 such that 17k + 15 ≡ 6 (mod 11). Since 17 ≡ 6 (mod 11), this gives 6k ≡ −9 ≡ 2 (mod 11); the inverse of 6 mod 11 is 2 (since 6×2=12≡1), so k ≡ 4 (mod 11). Taking k = 4: S = 17(4) + 15 = 83.

**Digital signature S** = 83

This signature S = 83 is attached to (or sent alongside) the message M = 8. Anyone receiving the pair (M, S) can independently verify authenticity using only the sender's public key, as shown next.

### 3.7 Step 4: Verifying the Signature Using the Public Key

Verification reverses the signing step using the public exponent e: the verifier computes M′ = S^e mod n and checks whether the result equals the originally received message M.

**M′ = S^e mod n** = 83^7 mod 187

Using CRT with the previously found residues Sp = 6 (mod 11) and Sq = 15 (mod 17):

Mp′ = $Sp^7$ mod 11 = $6^7$ mod 11: $6^2 = 36 \bmod 11 = 3$; $6^4 = 3^2 = 9$; $6^7 = 6^4 \times 6^2 \times 6^1 = 9 \times 3 \times 6 \bmod 11 = 162 \bmod 11 = 8$ → Mp′ = 8

$Mq' = Sq^7 \bmod 17 = 15^7 \bmod 17$. Since $15 \equiv -2 \pmod{17}$: $(-2)^7 = -128 \bmod 17 = -128 + 136 = 8 \;\rightarrow\; Mq' = 8$

Both partial results agree: $Mp' = 8$ and $Mq' = 8$, so the recombined result modulo n = 187 is:

**Recovered value M′ =** 8

### Verification Outcome

**Result:** M′ = 8 = M (the original message) ✓

Because the verification computation exactly reproduces the original message, the signature is confirmed valid. If even a single bit of the message or signature had been altered in transit, the recomputed M′ would not equal M, and verification would fail — this sensitivity to any change is precisely what gives digital signatures their integrity guarantee.

#### 3.7.1 Demonstrating Integrity: Detecting a Tampered Message

To make the integrity guarantee concrete, consider what happens if an attacker intercepts the pair (M = 8, S = 83) and alters the message to M = 9 before forwarding it, without access to the private key needed to produce a new, matching signature. The receiver still verifies using the original signature S = 83 and the public key, computing M′ = 83^7 mod 187 = 8, exactly as in Step 4 above. The receiver then compares this recomputed value against the received message:

| Quantity | Value | Outcome |
|---|---|---|
| Received (tampered) message | 9 | — |
| Recomputed M′ = S^e mod n | 8 | — |
| Match? | $9 \neq 8$ | Verification FAILS — tampering detected |

Because the recomputed value (8) does not match the received message (9), the receiver immediately knows the message has been altered in transit, and rejects it. This is the practical mechanism by which RSA signatures protect integrity: any change to the message — however small — breaks the match between the received message and the value recovered from the signature, without requiring any separate checksum or hash comparison step.

## 3.8 Step 5: How Digital Signatures Ensure Non-Repudiation

#### 3.7.1 The Core Argument

Non-repudiation means a signer cannot later deny having signed a message. RSA signatures achieve this because generating a valid signature S = M^d mod n requires knowledge of the private key d, and d is (and must remain) known only to the legitimate signer. Since verification only requires the public key (e, n), any third party — a recipient, an auditor, or a court — can

confirm that a signature could only have been produced by the party holding the corresponding private key, without that party needing to reveal the private key itself.

### 3.7.2 Supporting Properties

- Authentication: successful verification proves the message originated from the holder of the private key, not an impersonator.
- Integrity: because the signature is a function of the exact message content, any tampering with M after signing causes verification to fail, so the receiver knows the message was not modified in transit.
- Binding: the signature cannot be transplanted onto a different message, since S was computed specifically from M using modular exponentiation, not as an independent token.

### 3.7.3 Real-World Applications

| **Application** | **Role of Digital Signatures** |
|---|---|
| TLS/SSL Certificates | Certificate Authorities sign public keys, letting browsers verify website authenticity |
| Code Signing | Software vendors sign executables so users can verify the code has not been tampered with |
| E-Government / Legal Documents | Digitally signed contracts and tax filings carry the same legal weight as handwritten signatures |
| Cryptocurrency Transactions | Transactions are signed with the sender's private key to prove authorization of funds |
| Email (S/MIME, PGP) | Signed emails let recipients verify sender identity and message integrity |

### 3.7.4 Comparison: Encryption vs. Digital Signature Use of Keys

| **Operation** | **Confidentiality (Encryption)** | **Non-repudiation (Signature)** |
|---|---|---|
| Sender uses | Recipient's public key | Sender's own private key |
| Recipient uses | Recipient's own private key | Sender's public key |
| Protects against | Eavesdropping | Denial of authorship / tampering |
| Goal | Only intended recipient can read | Anyone can verify who signed |

### 3.8.4 Legal Recognition of Digital Signatures

The non-repudiation property demonstrated mathematically above has direct legal weight in many jurisdictions. Laws such as the U.S. ESIGN Act and UETA, the EU's eIDAS Regulation, and India's Information Technology Act all recognise properly implemented digital signatures as legally binding, on the basis that the underlying cryptography provides the same assurance of authorship and intent as a handwritten signature — and in many respects stronger assurance,

since a forged handwritten signature can often be produced with modest skill, whereas forging an RSA signature without the private key is computationally infeasible for adequately sized keys. This is why digitally signed contracts, e-filed tax returns, and notarised electronic documents are treated as admissible evidence of agreement in courts that recognise these frameworks.

### 3.8.5 A Known Weakness: Existential Forgery in Textbook RSA

It is worth noting explicitly that the simplified 'sign the raw message' scheme demonstrated in this question, while mathematically illustrative, has a well-documented weakness when used exactly as shown. Because RSA is multiplicatively homomorphic — that is, S1 × S2 mod n is a valid signature on M1 × M2 mod n whenever S1 signs M1 and S2 signs M2 — an attacker who can obtain signatures on two chosen messages can algebraically combine them into a valid forged signature on a third message they never asked to be signed, without ever learning the private key d. This is called an existential forgery attack. Real-world signature standards defeat this by first hashing the message with a cryptographically secure hash function before signing (as discussed in Section 3.3.2), which destroys the multiplicative relationship an attacker would need to exploit.

### 3.8.6 Standards Built on This Principle

| Standard | Underlying Signature Scheme | Typical Use |
|---|---|---|
| PKCS#1 v1.5 / RSA-PSS | RSA with hash-then-sign padding | TLS certificates, S/MIME, code signing |
| DSS (Digital Signature Standard) | DSA / ECDSA | US federal government signatures |
| ECDSA | Elliptic Curve variant of DSA | Bitcoin, TLS 1.3, modern mobile devices |
| EdDSA | Edwards-curve signatures | SSH keys, modern high-performance systems |

## 3.9 Additional Cross-Check: Direct Binary (Square-and-Multiply) Verification

Step 4 can also be verified directly, without the CRT shortcut, using the square-and-multiply method on the public exponent e = 7. Since 7 in binary is simply 111 (3 bits), this is a short but instructive confirmation:

**7 in binary** = 1 1 1

| Bit # | Bit value | Running value after squaring | Running value after multiply (if bit=1) |
|---|---|---|---|
| 1 (MSB) | 1 | $1^2 \bmod 187 = 1$ | $1 \times 83 \bmod 187 = 83$ |

| | | | |
|---|---|---|---|
| 2 | 1 | $83^2 \bmod 187 = 157$ | $157 \times 83 \bmod 187 = 128$ |
| 3 (LSB) | 1 | $128^2 \bmod 187 = 115$ | $115 \times 83 \bmod 187 = 8$ |

**Final result:** 83^7 mod 187 = 8

This confirms, through an entirely independent computational path from the CRT method used in Step 4, that the verified message $M' = 8$ is correct and equal to the original message $M = 8$ — giving strong confidence that both the signature $S = 83$ and the key pair ($d = 23$, $e = 7$) were computed correctly.

### 3.10 Summary of Results — Question 3

| **Quantity** | **Value** |
|---|---|
| n | 187 |
| φ(n) | 160 |
| Public key (e, n) | (7, 187) |
| Private key (d, n) | (23, 187) |
| Original message M | 8 |
| Digital signature S | 83 |
| Verified message M′ | 8 (matches original ✓ — signature valid) |

### 3.11 Conclusion

This exercise demonstrated the full RSA digital signature lifecycle: key generation, signature creation using the private key, and signature verification using the public key. The recovered value $M' = 8$ exactly matched the original message $M = 8$, confirming the signature's validity. More broadly, the exercise illustrates why digital signatures are indispensable in modern secure systems: they bind a specific identity (via a private key only that party possesses) to a specific piece of data in a way that any outside observer can verify but no outside observer can forge — the mathematical basis for non-repudiation in digital communications.

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 20% | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection | 20% | Selects most appropriate method/ formula with strong justification | Selects correct method with minor errors | Inappropriate or partially correct method | Unable to select method |
| Solution Process | 25% | Logical, systematic steps with correct approach throughout | Mostly correct steps with minor mistakes | Disorganized steps; multiple errors | No clear process |
| Accuracy of Calculation | 20% | All calculations accurate; correct final answer | Minor calculation errors | Frequent errors; incorrect final answer | No valid calculations |
| Interpretation of Results | 15% | Clearly interprets results with units and engineering relevance | Basic interpretation with minor omissions | Limited or unclear interpretation | No interpretation |